CODER BUDDY

Local LLM-Based Automated Detection and Correction of Secure Coding Violations and AI-Powered Project Generation

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for

SOFTWARE ENGINEERING (CA 725)

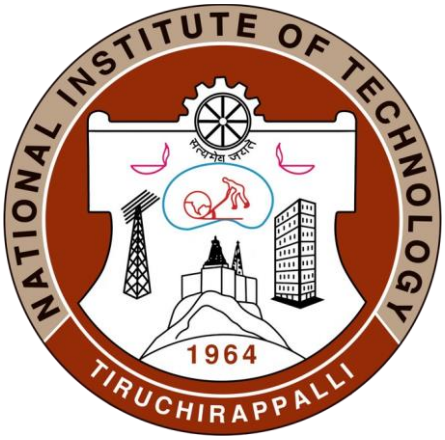
Submitted by

Sanket Yadav — 205125094

Himanshu Choudhary — 205125040

Under the Guidance of

Ms. Rajalakshmi Ma'am



NATIONAL INSTITUTE OF TECHNOLOGY

TIRUCHIRAPPALLI – 620015

DEPARTMENT OF COMPUTER APPLICATIONS

October 2026

DECLARATION

We declare that the project report entitled “Coder Buddy: An AI-Powered Project Generator and Secure-Coding Scanner and Fixer” is submitted to the Department of Computer Applications, National Institute of Technology Tiruchirappalli, in partial fulfilment of the requirements for Software Engineering (CA 725).

Place: NIT Tiruchirappalli

Date: 28/10/2026

Sanket Yadav
Roll No.: 205125094

Department of Computer Applications
NIT Tiruchirappalli

Himanshu Choudhary
Roll No.: 205125040

Department of Computer Applications
NIT Tiruchirappalli

CERTIFICATE

This is to certify that the project report entitled “Coder Buddy: An AI-Powered Project Generator and Secure-Coding Scanner and Fixer”, submitted by Sanket Yadav Roll no. 205125094 and Himanshu Choudhary Roll no. 205125040 to the Department of Computer Applications, National Institute of Technology Tiruchirappalli, is a record of project work submitted in partial fulfilment of the requirements for Software Engineering (CA 725).

Ms. Rajalakshmi ma’am	Dr. S. Dominic
Project Guide Department of Computer Applications	Head of the Department Department of Computer Applications

ACKNOWLEDGEMENTS

We express our sincere gratitude to Ms. Rajalakshmi Ma'am, Department of Computer Applications, National Institute of Technology Tiruchirappalli, for her guidance and support throughout the development of this project.

We are thankful to the Department of Computer Applications for providing the academic environment and opportunity to undertake this project as part of Software Engineering (CA 725).

We also acknowledge the contributions of the open-source software ecosystem and the developers of the libraries and tools used in the project. These tools supported the implementation of the multi-agent workflow, user interface, file operations, security analysis, and automated testing.

Sanket Yadav— 205125094

Himanshu Choudhary— 205125040

ABSTRACT

Software development involves several activities, including requirement analysis, system design, code implementation, and security testing. Performing these activities manually can require considerable time and effort, particularly when developers must repeatedly create project files and identify potential security weaknesses. Coder Buddy is a Python-based application designed to simplify project development through AI-assisted code generation and secure-coding analysis.

The project uses a multi-agent architecture consisting of Planner, Architect, and Coder agents. The Planner interprets the user's natural-language requirements and develops a project plan. The Architect organizes the plan into implementation tasks and determines the required files. The Coder executes these tasks using controlled file-management tools to generate the project. The application provides a Streamlit-based graphical interface and a command-line interface.

In addition to project generation, Coder Buddy provides a secure-coding scanner and automated correction functionality. The scanner combines deterministic security rules with optional large language model analysis to identify potential coding weaknesses. Findings can be associated with security references such as Common Weakness Enumeration (CWE), the OWASP Top 10, and CERT C/C++ guidelines, where applicable. The system validates model-generated findings against the source code, applies eligible corrections, checks syntax where supported, and rescans modified files to determine whether reported violations remain.

The application supports local model execution through Ollama and cloud-based model access through Groq. It also provides safeguards for uploaded source code, including archive validation and protection against unsafe file operations. The project was tested using a pytest-based automated test suite. The recorded test execution completed with 198 passed tests, 2 skipped tests, and 1 deselected test.

Coder Buddy demonstrates how AI-assisted development and security analysis can be integrated into a single application. However, its findings are not guaranteed to represent exploitable vulnerabilities, and its automated corrections require appropriate review. The application is intended to assist developers rather than replace professional security auditing or comprehensive static application security testing.

Keywords: Artificial Intelligence, Multi-Agent Systems, Code Generation, Secure Coding, Vulnerability Detection, Automated Code Correction, Python, LangGraph, Streamlit, Ollama.

TABLE OF CONTENTS

Abstract 5
List of Figures 7
List of Tables 8
List of Abbreviations 9
CHAPTER 1: Introduction 10
1.1 Background 10
1.2 Problem Statement 10
1.3 Objectives 11
1.4 Scope of the Project 11
1.5 Motivation 12
1.6 Proposed System 12
CHAPTER 2: Requirements Analysis and System Design 13
2.1 Requirements Analysis 13
2.2 Functional Requirements 13
2.3 Non-Functional Requirements 14
2.4 Feasibility Study 15
2.5 System Architecture 16
2.6 Module Description 17
2.7 Data Flow 18
2.8 Security Considerations 19
CHAPTER 3: Implementation and Testing 20
3.1 Technologies and Tools 20
3.2 Implementation of Project Generation 21
3.3 Implementation of Secure-Coding Analysis 21
3.4 Automated Code Correction 22
3.5 User Interface 23
3.6 Testing Methodology 23
3.7 Test Results 24
CHAPTER 4: Results, Conclusion and Future Scope 26
4.1 Results 26
4.2 Advantages 27
4.3 Limitations 28
4.4 Conclusion 28
4.5 Future Scope 29
References 31

LIST OF FIGURES

- Figure 2.1: Architecture of Coder Buddy
- Figure 3.1: Coder Buddy project-generation interface
- Figure 3.2: Model configuration and project generation in the Streamlit interface

LIST OF TABLES

- Table 2.1: System Functional Requirements Specification
- Table 2.2: System Non-Functional Requirements Specification
- Table 3.1: Technologies and Tools
- Table 3.2: Test Categories

LIST OF ABBREVIATIONS

Abbreviation	Full Description
AI	Artificial Intelligence
API	Application Programming Interface
CA	Course code used for Software Engineering
CLI	Command-Line Interface
CWE	Common Weakness Enumeration
HTML	HyperText Markup Language
LLM	Large Language Model
NIT	National Institute of Technology
OWASP	Open Worldwide Application Security Project
pytest	Python testing framework
REST	Representational State Transfer
UI	User Interface

CHAPTER 1: Introduction

1.1 Background

Software development is a structured process involving requirement identification, planning, architecture design, implementation, testing, and maintenance. Developers must translate user requirements into working applications while ensuring that the resulting code is understandable, maintainable, and secure.

Traditional development workflows often require developers to create project structures manually, determine the files required for each feature, implement the necessary functionality, and inspect the code for potential weaknesses. These activities can become repetitive and time-consuming, particularly when developing small applications, prototypes, or demonstration projects.

Artificial Intelligence (AI), particularly Large Language Models (LLMs), has introduced new approaches to software development. These models can interpret natural-language instructions, generate source code, explain programming concepts, and assist in identifying potential coding issues. When combined with workflow orchestration and controlled tools, AI models can be organized into multiple agents that perform distinct responsibilities.

However, generated source code may contain errors, incomplete functionality, insecure coding patterns, or unsuitable implementation choices. Therefore, code generation alone is insufficient for a reliable development workflow. A mechanism is also needed to inspect generated or existing code and help developers correct potential security problems.

Coder Buddy addresses these needs by combining AI-assisted project generation with secure-coding analysis and automated correction. It provides a unified interface through which users can describe a project, generate its initial structure, scan source code, review potential violations, and attempt eligible corrections.

1.2 Problem Statement

Developers frequently encounter two related challenges: the time required to transform application requirements into a structured codebase and the difficulty of consistently identifying security weaknesses during development.

Manual project creation requires developers to plan files, organize implementation tasks, and write repetitive code. Although AI-based code generation can reduce this effort, generated code may still contain insecure patterns or implementation errors.

Security analysis introduces another challenge. Developers must understand relevant coding guidelines, examine source files, identify potentially unsafe constructs, and verify that corrections do not introduce new problems. These activities may require specialized knowledge and additional tools.

There is therefore a need for an integrated system that can:

- Convert natural-language project descriptions into structured implementation plans.
- Generate project files through coordinated AI agents.
- Analyse generated or existing source code for potential secure-coding violations.
- Suggest or apply corrections while preserving safeguards around file operations.
- Validate corrections through source-code checks and subsequent scans.
- Present results through an accessible graphical interface and a command-line interface.

Coder Buddy is developed to address these requirements through an AI-assisted project generation and secure-coding workflow.

1.3 Objectives

The primary objective is to develop an integrated application that supports automated project generation and secure-coding analysis.

The specific objectives are:

1. To accept project requirements expressed in natural language.
2. To divide project generation into planning, architecture, and coding stages.
3. To generate project files through controlled file-management tools.
4. To provide a graphical interface for project generation and code scanning.
5. To support both cloud-based and locally executed language models.
6. To identify potential security weaknesses using deterministic rules and optional LLM-based analysis.
7. To associate applicable findings with established security references.
8. To validate model-generated findings against actual source-code content.
9. To apply eligible corrections with safeguards and preserve recoverable copies of modified files.
10. To rescan modified code and determine whether identified violations remain.
11. To provide machine-readable and human-readable scan reports.
12. To evaluate the application using automated tests.

1.4 Scope of the Project

The scope of Coder Buddy includes the following components.

AI-assisted project generation: The system interprets a natural-language prompt and produces a project plan, an ordered implementation plan, and the corresponding source files.

Multi-agent coordination: The project generation process is divided among Planner, Architect, and Coder agents to separate responsibilities and organize implementation tasks.

Secure-coding analysis: The system examines source code using deterministic rules and optional LLM-based analysis. Supported languages include Python, JavaScript, TypeScript, HTML, C, and C++, with additional languages handled through applicable generic rules and model-based analysis.

Automated correction: Eligible violations can be corrected through controlled patching, validation, backup creation, and rescanning.

Multiple interfaces: Users can interact with the system through a Streamlit graphical interface or a command-line workflow.

Model flexibility: The system supports local inference through Ollama and cloud-based inference through Groq.

Security safeguards: The system incorporates protections for uploaded archives, source-code handling, file modifications, and model-generated findings.

The project does not guarantee that all vulnerabilities will be detected or that all corrections will be safe in every execution environment. Its findings and patches should be reviewed before deployment in production systems.

1.5 Motivation

The motivation behind Coder Buddy is to reduce the effort involved in repetitive development tasks while encouraging secure coding practices.

AI-assisted development tools can generate code quickly, but the quality and security of generated code remain important concerns. An application that combines generation and security analysis can help developers examine potential weaknesses earlier in the development process.

The project also explores the use of multi-agent systems, in which different agents perform specialized tasks rather than relying on one undivided generation step. This separation can improve workflow organization and make the responsibilities of individual stages easier to understand.

Another motivation is the availability of local language models. By supporting Ollama, Coder Buddy allows users to perform model inference locally, subject to the selected model's capabilities and configuration. Cloud-based inference remains available as an alternative.

1.6 Proposed System

Coder Buddy consists of two major functional areas: project generation and secure-coding analysis.

During project generation, the user enters a natural-language description. The Planner converts the description into a structured plan. The Architect converts that plan into implementation tasks, including the files required for the project. The Coder processes these tasks and uses file-management tools to create the source files.

During secure-coding analysis, the user can scan code generated by the application or provide an existing project for examination. Deterministic security rules identify known coding patterns, while optional LLM analysis can examine additional context. Model-generated findings are checked against the source code before being accepted.

If automatic correction is enabled, the system attempts to generate and validate patches for eligible findings. Modified files are backed up, applicable syntax checks are performed, and the files are rescanned. The process repeats within a configured correction-round limit.

The application presents the generated project information, scan findings, and available reports through its user interface and command-line functionality.

CHAPTER 2: Requirements Analysis And System Design

2.1 Requirements Analysis

Requirements analysis identifies the functionality and quality characteristics required from the proposed application.

Coder Buddy must support project generation, file management, secure-coding analysis, optional automated correction, report generation, and interaction with different LLM providers. It must also protect user files and avoid treating unverified model-generated findings as confirmed vulnerabilities.

The system is implemented primarily in Python and uses a multi-agent workflow for project generation. Security analysis is handled through a combination of deterministic rules and optional model-assisted analysis.

2.2 Functional Requirements

The functional requirements describe the operations that the system must perform.

ID	Functional requirement	Description
FR1	Accept project requirements	The system accepts natural-language descriptions of the desired application.
FR2	Plan the project	The Planner produces a structured project plan.
FR3	Design implementation tasks	The Architect determines the implementation sequence and required files.
FR4	Generate source files	The Coder creates files using controlled file-management tools.
FR5	Support user interfaces	The application provides Streamlit and command-line interaction.
FR6	Scan source code	The system examines generated or existing source code for potential violations.
FR7	Apply security rules	Deterministic rules identify applicable insecure coding patterns.
FR8	Perform model-assisted analysis	The system can use an LLM to identify additional potential issues.
FR9	Validate findings	Model-generated findings are checked against actual source-code content.

FR10	Correct eligible violations	The system can generate and apply validated corrections when automatic fixing is enabled.
FR11	Verify corrections	Modified files are rescanned to determine whether the reported violations remain.
FR12	Generate reports	The system provides scan information in JSON and Markdown formats.
FR13	Support model providers	The system supports configured Groq and Ollama providers.
FR14	Protect source files	Existing projects can be scanned using a copy so that the original files remain untouched by the scan-and-fix workflow.

2.3 Non-Functional Requirements

ID	Requirement	Description
NFR1	Usability	The graphical interface should make project generation and scanning accessible to users.
NFR2	Maintainability	The application should separate workflow, security rules, file operations, and interface functionality into modules.
NFR3	Reliability	Invalid or malformed model outputs should be handled through validation and appropriate error-handling mechanisms.
NFR4	Security	File operations and code modifications should be restricted and validated.
NFR5	Portability	The Python application should run in a compatible environment with the required dependencies.
NFR6	Model flexibility	The application should support configured local and cloud-based LLM providers.

NFR7	Traceability	Security findings should provide useful information about the affected code and applicable security references.
NFR8	Recoverability	The correction workflow should preserve backups before modifying files.
NFR9	Testability	The application should support automated testing of its major components.
NFR10	Transparency	Scan findings and correction results should be available for review rather than being treated as unquestionable security guarantees.

2.4 Feasibility Study

2.4.1 Technical Feasibility

The project is technically feasible using Python and its supporting libraries. LangGraph provides workflow orchestration for the multi-agent generation process, while Streamlit supports the graphical interface.

The secure-coding component combines deterministic rules with LLM-based analysis. Ollama provides access to local models, while Groq supports cloud-based inference. The system also uses automated testing tools to validate its implemented functionality.

The application can be executed on a compatible computer with Python and the necessary dependencies. Local model execution additionally requires a suitable model installation and sufficient system resources.

2.4.2 Economic Feasibility

The project uses Python and several open-source libraries, which reduces the initial software licensing requirements. Local inference through Ollama can reduce dependence on paid cloud inference, although hardware usage and electricity costs remain relevant.

Cloud inference through Groq may be subject to provider-specific access limits, usage quotas, or pricing. The overall cost therefore depends on the selected model provider, hardware configuration, and frequency of use.

2.4.3 Operational Feasibility

The application offers a graphical interface for users who prefer interactive operation and a command-line interface for terminal-based workflows. This makes the system usable for demonstrations, development experiments, and code-review assistance.

However, users should understand the limitations of automated security analysis and review proposed changes before relying on them in production environments.

2.5 System Architecture

Coder Buddy follows a modular architecture in which project generation and secure-coding analysis are connected through the application interface.

2.5.1 Architecture Components

1. User Interface Layer

The user interacts with the application through Streamlit or the command-line interface. The user provides a project prompt, selects relevant settings, initiates generation or scanning, and reviews the results.

2. Agent Orchestration Layer

LangGraph coordinates the project generation workflow. The workflow organizes planning, architecture, coding, and relevant state transitions.

3. Planner Agent

The Planner interprets the user's requirements and generates a structured plan describing the intended project.

4. Architect Agent

The Architect converts the project plan into an ordered set of implementation tasks. Each task identifies work required to construct part of the application.

5. Coder Agent

The Coder executes the implementation tasks. It uses controlled tools such as `read_file`, `write_file`, `list_files`, and `get_current_directory` to interact with project files.

6. LLM Provider Layer

The system supports configured model providers, including Groq for cloud inference and Ollama for local inference. The selected provider is used according to the operation and application configuration.

7. Secure-Coding Analysis Layer

The security layer combines deterministic rules and optional LLM-based analysis. Findings are validated against the actual source code before being accepted.

8. Correction and Verification Layer

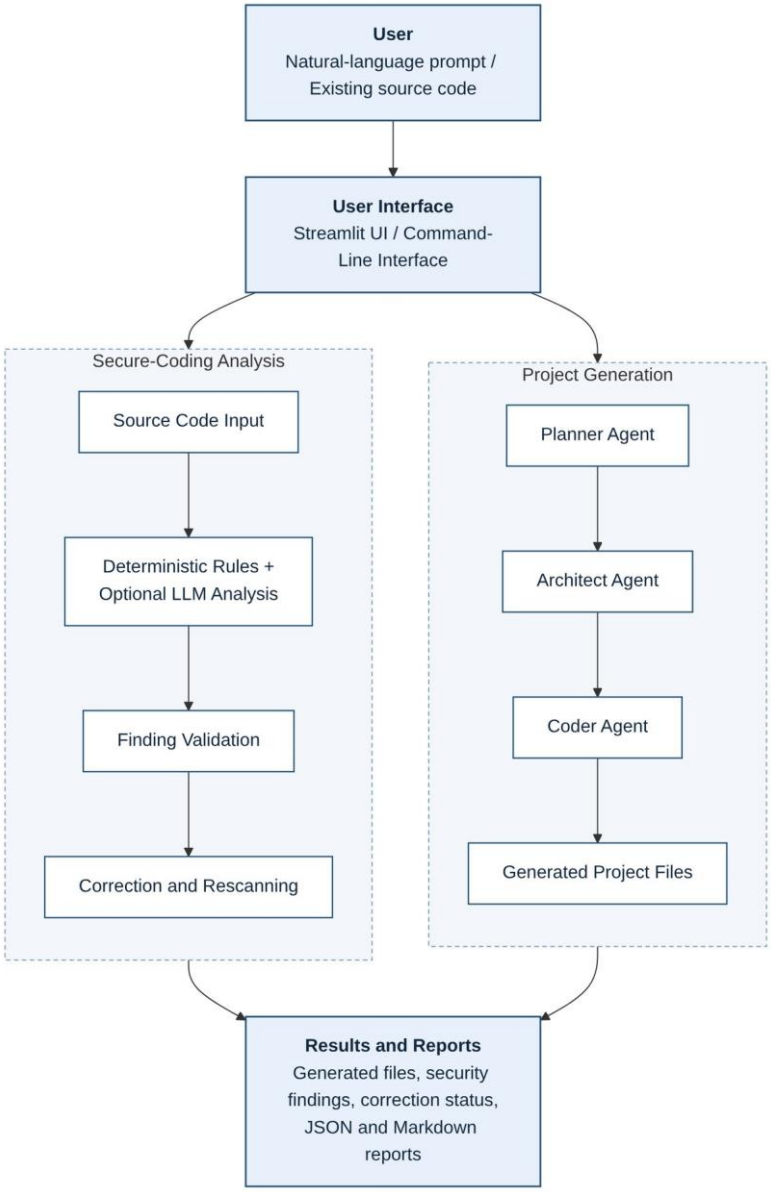
Eligible corrections are checked before application. The system preserves backups, performs applicable syntax validation, applies patches, and rescans modified files.

9. Reporting Layer

The application produces scan reports in JSON and Markdown formats, providing information about detected findings and correction outcomes.

2.5.2 Architecture Diagram

Figure 2.1: Architecture of Coder Buddy



The diagram represents the major functional stages of the system. The project generation workflow creates application files, while the secure-coding workflow examines generated or existing code. The results are presented to the user through the interface and reporting mechanisms.

2.6 Module Description

2.6.1 Agent Workflow Module

The agent workflow coordinates the project generation process. It manages the sequence of planning, architecture creation, and coding tasks. The workflow also handles state transitions and supports retry or fallback behaviour when model-generated tool outputs are malformed.

2.6.2 State Management Module

The project uses structured state models to represent information exchanged between agents. These models help maintain consistency between planning, implementation tasks, and generated outputs.

2.6.3 LLM Integration Module

The LLM integration module manages access to configured model providers. It supports local and cloud inference and provides the model interactions required by the generation and analysis workflows.

2.6.4 File Management Module

The file management module provides controlled operations for reading, writing, and listing files. Generated project files are restricted to the designated project workspace, helping prevent unintended file modifications outside the intended working area.

2.6.5 Security Rules Module

The security rules module contains deterministic checks for known insecure coding patterns. These rules provide predictable detection behaviour for supported patterns and can associate findings with relevant security references.

2.6.6 Secure-Coding Analysis Module

The secure-coding module coordinates rule-based checks and optional model-assisted analysis. It validates model-generated findings against source-code excerpts and line information before accepting them.

2.6.7 Correction Module

The correction module attempts to generate and validate patches for eligible findings. It preserves backups, applies changes through controlled operations, and verifies the result by rescanning the affected files.

2.6.8 Reporting Module

The reporting module records scan results and correction information in JSON and Markdown formats. These reports support further inspection of the findings and their status.

2.7 Data Flow

The application supports two main data flows.

2.7.1 Project Generation Data Flow

13. The user enters a project description.
14. The Planner interprets the description and creates a project plan.
15. The Architect converts the plan into ordered implementation tasks.
16. The Coder processes the tasks and generates project files.
17. The application displays the generated project information and available files.
18. If enabled, the generated code can be passed to the secure-coding workflow.

2.7.2 Secure-Coding Data Flow

19. The user selects generated code or supplies an existing project.

20. The application prepares the source code for analysis.
21. Deterministic security rules examine the relevant files.
22. Optional LLM analysis identifies additional potential issues.
23. Model-generated findings are validated against the source code.
24. Accepted findings are presented to the user.
25. If automatic correction is enabled, eligible findings are passed to the correction workflow.
26. Patches are validated and applied with backup safeguards.
27. Modified files are rescanned.
28. The application records the results and generates reports.

2.8 Security Considerations

Security is an important part of the system because the application processes source code and may modify files automatically.

Source-code validation: Model-generated findings must correspond to actual code in the source files. This reduces the risk of accepting unsupported findings.

Controlled file operations: Project generation uses designated workspaces and controlled file tools.

Backup creation: The correction workflow preserves copies of files before applying modifications.

Patch validation: Proposed patches are checked before application, with syntax validation where supported.

Rescanning: Modified files are rescanned to determine whether the original violations remain.

Uploaded archive protection: The application incorporates checks for unsafe archive paths, symbolic links, and archive-size risks.

Prompt-injection mitigation: Source code is treated as untrusted input during scanning and correction. Scan and fix model calls are designed without general file-operation tools.

Local inference option: Ollama allows users to perform supported model operations locally, reducing the need to transmit source code to a cloud model provider. Cloud-based inference should be selected with awareness of applicable data-handling requirements.

These safeguards reduce certain operational risks but do not establish that every scan or correction is secure in every environment.

CHAPTER 3: Implementation And Testing

3.1 Technologies and Tools

The following technologies and tools are associated with the Coder Buddy implementation.

Technology or tool	Purpose
Python	Primary programming language
LangGraph	Orchestration of the multi-agent project generation workflow
Streamlit	Graphical user interface
Groq	Cloud-based LLM inference provider
Ollama	Local LLM execution
`openai/gpt-oss-120b`	Cloud model identified in the project configuration
`qwen2.5-coder:3b`	Local model shown in the supplied application screenshot and testing environment
Pydantic	Structured data models and state validation
Pytest	Automated testing
Git	Version control, if used in the development workflow
JSON	Structured scan-result reporting
Markdown	Human-readable scan reports
HTML, CSS and JavaScript	Languages supported by project-generation examples and relevant code-analysis rules
CWE	Classification of common software weaknesses
OWASP Top 10:2021	Reference for major web application security risks
CERT C/C++	Secure-coding guidance for C and C++
Windows	Operating environment shown in the supplied testing output

The AI tools directly associated with the application's model functionality are Groq and Ollama. The supplied application screenshot shows Ollama configured with `qwen2.5-coder:3b`. ChatGPT was also used in preparing this report. Other AI-assisted development tools should be listed only if they were actually used during implementation; their use cannot be established from the project files supplied here.

3.2 Implementation of Project Generation

The project generation component converts natural-language requirements into a structured project implementation.

3.2.1 Input Processing

The user describes the desired application in a text prompt. For example, the supplied screenshot shows the prompt:

“Create a colourful landing page for a coffee shop in html, css and js.”

The prompt identifies the expected application and its principal technologies. The system passes the requirements through the agent workflow.

3.2.2 Planning

The Planner interprets the user's request and prepares a structured plan describing the intended project. This provides a basis for the subsequent architecture and coding stages.

3.2.3 Architecture Generation

The Architect transforms the project plan into an ordered collection of implementation tasks. These tasks specify the work required to create the project files.

3.2.4 Code Generation

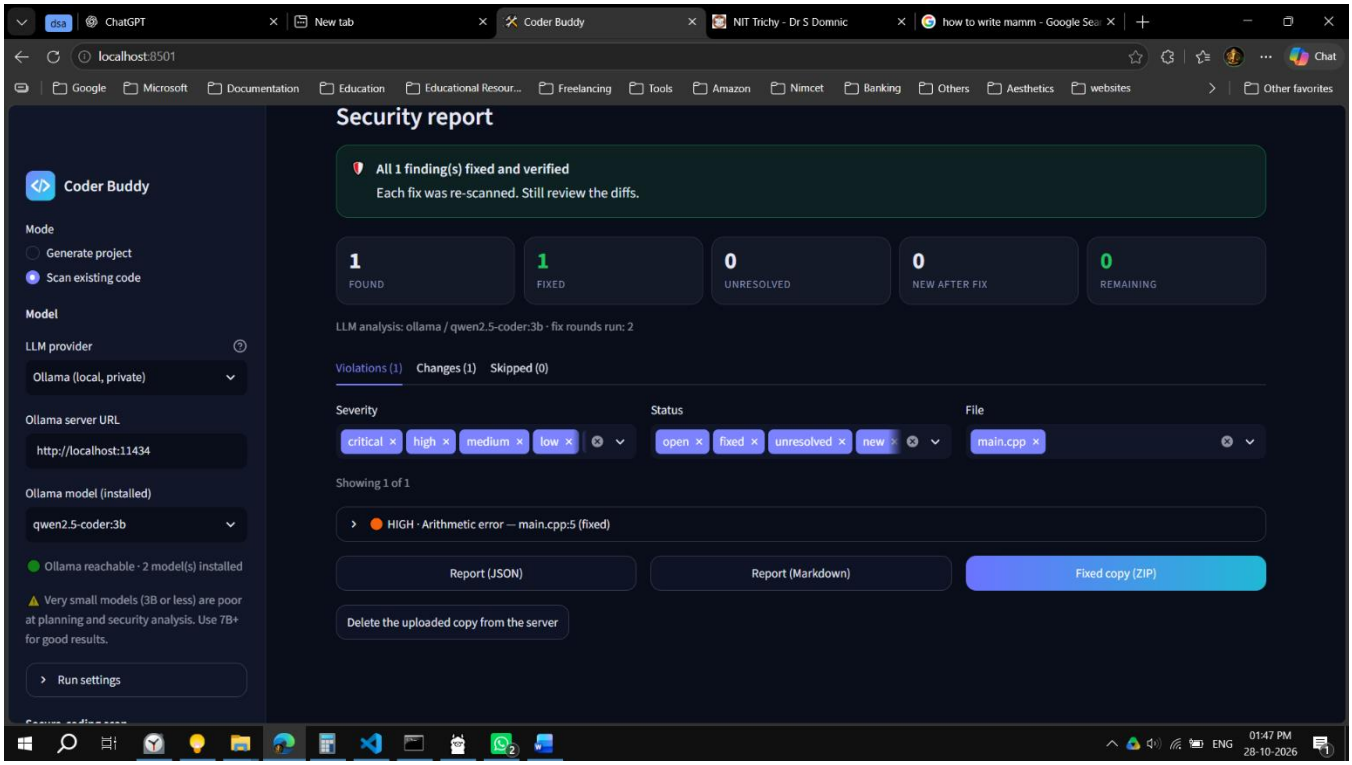
The Coder processes the implementation tasks and uses controlled file-management tools to create source files. The generated project is stored in the designated project workspace.

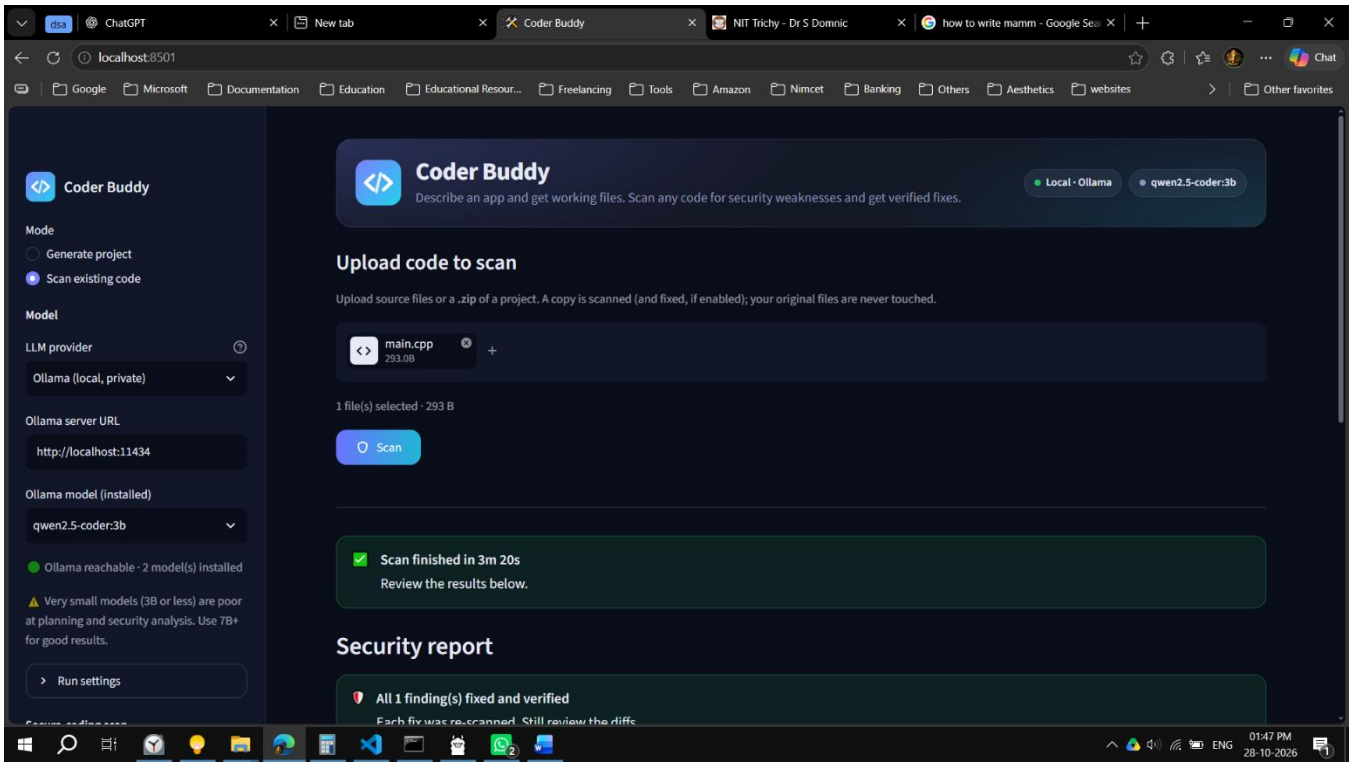
3.2.5 Workflow Monitoring

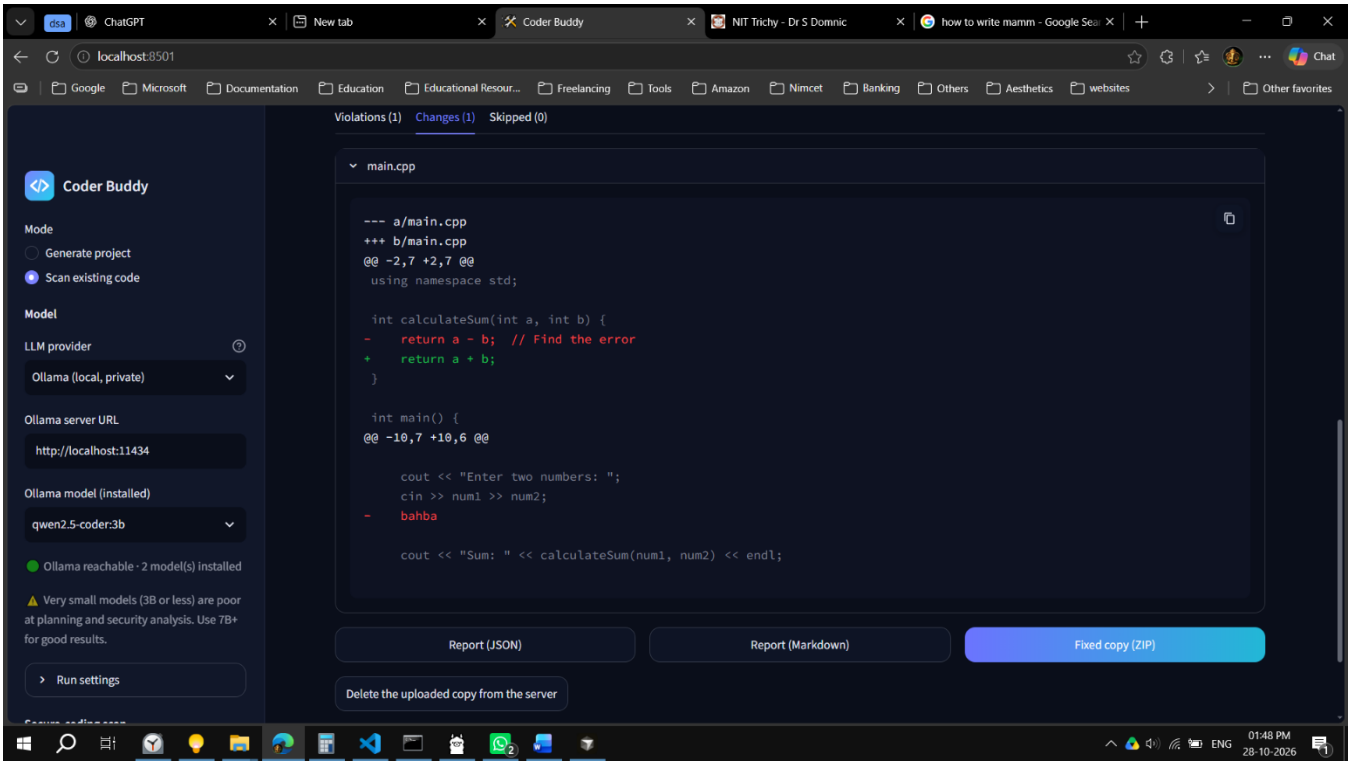
The application displays the completion status of the Planner, Architect, and Coder stages. In the supplied screenshot, all three stages display a completed status for the coffee shop landing-page example.

This demonstrates a successful completion of the displayed generation workflow, but it does not independently establish that every generated file is functionally correct or free from security weaknesses.

Figure 3.1: Coder Buddy project-generation interface


ChatGPT
New tab
Coder Buddy
NIT Trichy - Dr S Domnic
how to write mamm - Google Sear
localhost:8501
Chat
Google
Microsoft
Documentation
Education
Educational Resour...
Freelancing
Tools
Amazon
Nimcet
Banking
Others
Aesthetics
websites
Other favorites
Coder Buddy
Mode
Generate project
Scan existing code
Model
LLM provider
Ollama (local, private)
Ollama server URL
http://localhost:11434
Ollama model (installed)
qwen2.5-coder:3b
Ollama reachable · 2 model(s) installed
Very small models (3B or less) are poor at planning and security analysis. Use 7B+ for good results.
Run settings
Security report
All 1 finding(s) fixed and verified
Each fix was re-scanned. Still review the diffs.
1
FOUND
1
FIXED
0
UNRESOLVED
0
NEW AFTER FIX
0
REMAINING
LLM analysis: ollama / qwen2.5-coder:3b · fix rounds run: 2
Violations (1)
Changes (1)
Skipped (0)
Severity
critical
high
medium
low
Status
open
fixed
unresolved
new
File
main.cpp
Showing 1 of 1
HIGH · Arithmetic error — main.cpp:5 (fixed)
Report (JSON)
Report (Markdown)
Fixed copy (ZIP)
Delete the uploaded copy from the server
ENG
01:47 PM
28-10-2026


ChatGPT
New tab
Coder Buddy
NIT Trichy - Dr S Domnic
how to write mamm - Google Sear
localhost:8501
Chat
Google
Microsoft
Documentation
Education
Educational Resour...
Freelancing
Tools
Amazon
Nimcet
Banking
Others
Aesthetics
websites
Other favorites
Coder Buddy
Mode
Generate project
Scan existing code
Model
LLM provider
Ollama (local, private)
Ollama server URL
http://localhost:11434
Ollama model (installed)
qwen2.5-coder:3b
Ollama reachable · 2 model(s) installed
Very small models (3B or less) are poor at planning and security analysis. Use 7B+ for good results.
Run settings
Coder Buddy
Describe an app and get working files. Scan any code for security weaknesses and get verified fixes.
Local · Ollama
qwen2.5-coder:3b
Upload code to scan
Upload source files or a .zip of a project. A copy is scanned (and fixed, if enabled); your original files are never touched.
main.cpp
293.0B
1 file(s) selected · 293 B
Scan
Scan finished in 3m 20s
Review the results below.
Security report
All 1 finding(s) fixed and verified
ENG
01:47 PM
28-10-2026



3.1 Implementation of Secure-Coding Analysis

The secure-coding component examines source code for potential weaknesses. It combines deterministic rules with optional LLM-based analysis.

3.1.1 Deterministic Rule-Based Analysis

Deterministic rules identify known patterns associated with potentially insecure coding practices. Because these checks follow defined conditions, they can provide predictable results for the patterns they cover.

The rule-based approach is useful for identifying known issues without requiring every finding to be generated by a language model. However, its coverage depends on the available rules and their implementation.

3.1.2 LLM-Based Analysis

An optional LLM examines source code to identify additional potential security issues. Model-assisted analysis can consider contextual patterns that may not be covered by a specific deterministic rule.

However, language models can produce inaccurate findings or suggest inappropriate corrections. Consequently, model output is not accepted without validation.

3.1.3 Finding Validation

The system validates model-generated findings by requiring quoted code to match the actual source file. It also checks applicable line-range information.

This validation helps reject findings that refer to code not present in the scanned source. It does not prove that every accepted finding is exploitable or that every reported weakness is security-critical.

3.1.4 Security References

Where applicable, findings can be associated with references such as:

- Common Weakness Enumeration (CWE).
- OWASP Top 10:2021.
- CERT C/C++ secure-coding guidelines.

These references provide a structured way to describe and classify potential weaknesses. Their availability and relevance depend on the rule or finding being reported.

3.2 Automated Code Correction

The correction component attempts to address eligible security findings.

The correction workflow includes the following stages:

29. Select a finding eligible for automatic correction.
30. Generate a proposed patch.
31. Validate the patch against the intended source-code change.
32. Perform applicable syntax checks.
33. Preserve a backup of the file before modifying it.
34. Apply the validated patch through controlled file operations.
35. Rescan the modified file.
36. Record whether the original violation remains.

When scanning an existing project, the workflow uses a copy so that the original source files remain untouched by the scan-and-fix process.

The application uses a configurable correction-round limit. The project configuration described in the README uses a default of two rounds, with a maximum of five.

The system treats a violation as fixed only when the subsequent scan no longer reports that violation. This is an operational verification mechanism rather than proof that the entire file or application is free from vulnerabilities.

3.3 User Interface

Coder Buddy provides a Streamlit-based interface and a command-line interface.

The Streamlit interface includes controls for selecting the operating mode, configuring the LLM provider, specifying the model, entering a project prompt, and enabling relevant security options.

The supplied screenshot shows the following configuration:

- Mode: Generate project.
- LLM provider: Ollama (local).
- Model: `qwen2.5-coder:3b`.
- Ollama server URL: `http://localhost:11434`.
- Recursion limit: 100.
- Clear previous generated files first: Enabled.
- Run secure-coding scan after generation: Disabled.
- Fix violations automatically: Enabled.

The screenshot also displays the completed Planner, Architect, and Coder stages and a generated project titled “Coffee Shop Landing Page.”

These settings represent the configuration shown in that particular screenshot and should not be interpreted as the mandatory settings for every execution.

Figure 3.2: Model configuration and project generation in the Streamlit interface

3.4 Testing Methodology

Testing was conducted using the pytest framework in a Windows environment with Python 3.11.15.

The test suite covers application startup behaviour, command-line functionality, graph routing, LLM parsing, security rules, finding validation, application state, and uploaded-file safety.

The following command was executed:

```
pytest
```

The recorded output indicates that 201 tests were collected, with one deselected and 200 selected for execution. The execution completed with 198 passed tests and 2 skipped tests.

3.4.1 Test Categories

Test module	Purpose
`test_app_smoke.py`	Basic application smoke tests
`test_cli.py`	Command-line behaviour
`test_graph_routing.py`	Agent workflow routing
`test_llm_parsing.py`	Parsing and handling of LLM outputs

test_rules.py	Deterministic security-rule checks
test_secure_validation.py	Security finding and correction validation
test_states.py	State-model behaviour
test_upload_safety.py	Uploaded-file and archive safety

The test suite exercises important internal components and application workflows. However, the number of passed tests alone does not establish complete security coverage, production readiness, or the accuracy of every LLM-generated correction.

3.5 Test Results

The recorded test execution produced the following output:

```
platform win32 -- Python 3.11.15
Python 3.11.15
pytest-9.1.1

collected 201 items / 1 deselected / 200 selected

tests\test_app_smoke.py        6 passed
tests\test_cli.py             15 passed
tests\test_graph_routing.py   21 passed
tests\test_llm_parsing.py     34 passed
tests\test_rules.py           19 passed
tests\test_secure_validation.py
                              38 passed, 2 skipped
tests\test_states.py          24 passed
tests\test_upload_safety.py   30 passed

198 passed, 2 skipped, 1 deselected
in 6.28 seconds
```

The output above reproduces the recorded overall result and test-module progress in a readable format. The final aggregate figures are taken from the supplied terminal log.

3.5.1 Summary of Automated Testing

198	2	1
Passed	**Skipped**	**Deselected**

Skipped

Deselected

Execution time: 6.28 seconds. Environment: Windows, Python 3.11.15, pytest 9.1.1.

The test results indicate that the selected automated test suite completed successfully, with 198 tests passing and 2 tests skipped. One test was deselected and was not part of the selected execution.

The tests cover several important areas, including workflow routing, output parsing, deterministic security rules, finding validation, state management, and upload safety. These results provide evidence of functional testing for the tested components.

The skipped tests were not executed, so their behaviour cannot be considered verified by this run. Similarly, the deselected test is outside the reported selected-test results. The test outcome should therefore be interpreted in the context of the suite's actual coverage.

Figure 3.3: Automated test execution using pytest

CHAPTER 4: Results, Conclusion And Future Scope

4.1 Results

The project resulted in an integrated Python application for AI-assisted project generation and secure-coding analysis.

4.1.1 Project Generation

The application successfully completed the Planner, Architect, and Coder stages in the supplied coffee shop landing-page example. The interface displayed the project title, description, technology stack, and completed status for all three agents.

This demonstrates the intended multi-agent workflow in the displayed execution.

4.1.2 Secure-Coding Analysis

The application implements a hybrid analysis approach that combines deterministic security rules with optional LLM-based analysis. Findings can be checked against the source code before being accepted.

The design reduces reliance on unvalidated model output, although the effectiveness of detection remains dependent on rule coverage, model behaviour, and validation quality.

4.1.3 Automated Correction

The correction workflow supports validated patch application, backup creation, syntax checks where applicable, and rescanning of modified files. The system can repeat the correction process within a configurable limit.

This provides a structured mechanism for attempting to resolve eligible findings. It does not establish that every patch will be correct or that a successful rescan proves the absence of all security weaknesses.

4.1.4 Automated Testing

The recorded pytest run completed with 198 passed tests, 2 skipped tests, and 1 deselected test in 6.28 seconds.

The results demonstrate that the selected tests for application smoke behaviour, command-line operations, workflow routing, parsing, rules, security validation, state management, and upload safety completed with the reported outcome.

4.1.5 Overall Outcome

The project integrates project generation and secure-coding assistance into a single application. It provides a modular workflow, multiple interfaces, model-provider flexibility, safeguards for source-code modification, and automated test coverage.

The results support the feasibility of the implemented approach. Further testing with diverse real-world projects, independent security tools, and systematically measured correction outcomes would be required to evaluate its effectiveness more comprehensively.

4.2 Advantages

4.2.1 Reduced Manual Effort

The multi-agent generation workflow automates parts of project planning, architecture preparation, and source-file creation. This can reduce repetitive development work.

4.2.2 Modular Architecture

The Planner, Architect, and Coder agents have separate responsibilities. The modular design makes the workflow easier to understand and supports changes to individual components.

4.2.3 Integrated Security Analysis

Project generation and secure-coding analysis are available within the same application. Users can examine generated code or scan an existing project.

4.2.4 Hybrid Detection Approach

Combining deterministic rules with optional LLM-based analysis provides two complementary methods for identifying potential coding weaknesses.

4.2.5 Validation of Findings

Checking model-generated findings against actual source-code content helps reduce unsupported findings.

4.2.6 Automated Correction Support

The application can attempt corrections for eligible findings, preserve backups, and rescan modified files. This provides a more structured workflow than applying unverified model suggestions directly.

4.2.7 Local Model Support

Ollama enables local model execution when the required model and resources are available. This can be useful when source code should remain within the local environment.

4.2.8 Multiple Interfaces

The Streamlit interface supports interactive use, while the command-line interface supports terminal-based workflows.

4.2.9 Structured Reporting

JSON and Markdown reports provide machine-readable and human-readable representations of scan results.

4.2.10 Automated Testing

The pytest suite covers multiple application components, supporting repeatable validation of implemented behaviour.

4.3 Limitations

4.3.1 Incomplete Rule Coverage

Deterministic rules can identify only the patterns covered by their implementation. New vulnerabilities, complex data flows, and application-specific weaknesses may remain undetected.

4.3.2 Possible Inaccuracies in LLM Findings

Large language models may produce false positives, false negatives, or explanations that do not accurately describe the behaviour of the source code. Validation reduces some errors but cannot eliminate them completely.

4.3.3 No Guarantee of Exploitability Assessment

A detected pattern does not necessarily mean that a vulnerability is exploitable in the application's actual operating environment. Determining exploitability may require contextual analysis and expert review.

4.3.4 Risk of Incorrect Corrections

Automatically generated patches may alter program behaviour, introduce new defects, or fail to address the underlying security problem. Backup creation and rescanning reduce certain risks but do not eliminate them.

4.3.5 Dependence on Model Resources

Local model performance depends on hardware resources, model size, and inference speed. Cloud-based inference depends on network access and the availability and limits of the selected provider.

4.3.6 Language and Framework Coverage

The quality of analysis varies according to the available deterministic rules and the selected model's capabilities. Support for a programming language does not imply complete coverage of its vulnerabilities.

4.3.7 Limited Evaluation of Correction Quality

The supplied test output establishes the results of the automated test suite, but it does not provide a comprehensive quantitative measurement of vulnerability-detection accuracy, false-positive rate, false-negative rate, or the security correctness of applied patches.

4.3.8 Need for Human Review

The application is intended to assist development and security review. Developers should examine scan findings, inspect correction diffs, run relevant application tests, and review the resulting code before using it in production.

4.4 Conclusion

Coder Buddy was developed as an AI-assisted application that combines multi-agent project generation with secure-coding analysis and automated correction support.

The project generation workflow uses Planner, Architect, and Coder agents to transform natural-language requirements into structured implementation tasks and source files. The Streamlit interface and command-line functionality provide different ways to interact with the application.

The secure-coding component combines deterministic rules and optional LLM-based analysis. It validates model-generated findings against source-code content and supports controlled correction through patch validation, backup creation, applicable syntax checks, and rescanning.

The recorded automated test execution completed with 198 passed tests, 2 skipped tests, and 1 deselected test. This provides evidence of successful execution across the selected tests for several important application components.

The project demonstrates the practical integration of AI-assisted development and security-analysis workflows. At the same time, the limitations of deterministic rules, LLM-generated findings, and automatic correction must be considered. Coder Buddy should be treated as a development and security-assistance tool rather than a replacement for comprehensive security testing or expert review.

Overall, the project provides a foundation for further work in multi-agent software development, secure-code analysis, and automated correction workflows.

4.5 Future Scope

Several improvements can extend the capabilities of Coder Buddy.

4.5.1 Expanded Security Rule Catalogue

The deterministic rule catalogue can be extended to cover additional vulnerabilities, frameworks, programming languages, and secure-coding practices.

4.5.2 Integration with Established Security Tools

The system could integrate with established static-analysis and dependency-scanning tools to provide complementary findings and stronger coverage.

4.5.3 Improved Correction Evaluation

A systematic evaluation framework could measure correction success rates, regression frequency, false-positive rates, and false-negative rates using labelled datasets and controlled test projects.

4.5.4 Regression Testing After Corrections

The correction workflow could be extended to run project-specific unit tests, integration tests, and regression tests after applying patches.

4.5.5 Improved Finding Prioritisation

Findings could be prioritised using severity, confidence, exploitability indicators, and application context, provided these values are supported by reliable evidence.

4.5.6 Enhanced Reporting

Future versions could provide more detailed reports containing severity summaries, source-code excerpts, correction diffs, scan history, and comparative results before and after correction.

4.5.7 Better Model Selection

The application could support systematic model comparison based on inference time, resource consumption, finding accuracy, and correction quality.

4.5.8 Continuous Integration Support

Integration with continuous integration pipelines could allow projects to be scanned automatically during development. Such integration would require suitable configuration and controls to prevent unreviewed patches from being deployed.

4.5.9 Broader Project Validation

Future evaluation could use a larger collection of real-world projects and compare Coder Buddy's results with established security-analysis tools and expert assessments.

4.5.10 Improved User Guidance

The interface could provide more detailed explanations of findings, correction decisions, and validation outcomes to help users understand security issues and make informed decisions.

REFERENCES

[1] LangChain, “LangGraph Documentation.” Available: https://docs.langchain.com/oss/python/langgraph

[2] Streamlit, “Streamlit Documentation.” Available: https://docs.streamlit.io/

[3] Python Software Foundation, “Python 3 Documentation.” Available: https://docs.python.org/3/

[4] Pytest Developers, “pytest Documentation.” Available: https://docs.pytest.org/

[5] Ollama, “Ollama Documentation.” Available: https://docs.ollama.com/

[6] Groq, “Groq API Documentation.” Available: https://console.groq.com/docs/overview

[7] MITRE, “Common Weakness Enumeration (CWE).” Available: https://cwe.mitre.org/

[8] OWASP Foundation, “OWASP Top 10:2021.” Available: https://owasp.org/Top10/

[9] Software Engineering Institute, Carnegie Mellon University, “SEI CERT C Coding Standard.” Available: https://wiki.sei.cmu.edu/confluence/display/c

[10] OpenAI, “GPT-OSS.” Available: https://openai.com/index/introducing-gpt-oss/